文件，而是直接内存。

解决方案

解决内存碎片的方法有两种：内存压缩和内存整理。

内存压缩

内存压缩的原理是将一个内存页的内容压缩后放入另一个内存页，从而释放一个内存页以供使用。内存压缩的优点是速度快，因为它只需要在内存中进行数据的压缩和解压缩，而不需要进行磁盘I/O操作。内存压缩的缺点是压缩率不高，因为它只能压缩一部分内存页，而且压缩后的内存页仍然占用内存空间。内存压缩的适用场景是内存碎片不严重，且内存压缩的开销小于内存整理的开销的情况。

内存整理的原理是将一个内存区域的所有内存页重新分配，以消除内存碎片。内存整理的优点是能够释放大量内存空间，以满足内存需求。内存整理的缺点是速度慢，因为它需要进行大量的磁盘I/O操作，而且内存整理的过程中，内存分配和释放的操作会被暂停，从而影响系统的性能。内存整理的适用场景是内存碎片严重，且内存压缩的效果不佳的情况。

Linux内核通过内存压缩和内存整理来解决内存碎片问题。内存压缩通过将一个内存页的内容压缩后放入另一个内存页，从而释放一个内存页。内存整理通过将一个内存区域的所有内存页重新分配，以消除内存碎片。内存压缩和内存整理的选择取决于内存碎片的严重程度和内存压缩的效果。

解决 Thread 超高频创建

现代语言都有一个解决方案：线程池。

```
线程池
- 线程复用
- 任务队列
- 动态扩缩容
- 生命周期管理
- 资源限制
- 优雅关闭
```

```
线程池
- 线程复用
- 任务队列
- 动态扩缩容
- 生命周期管理
- 资源限制
- 优雅关闭
```

线程复用：线程池会预创建固定数量的线程，这些线程会长时间存在，Worker 线程不会频繁创建和销毁。

任务队列：当任务提交时，先放入任务队列，Worker 线程从队列取任务执行。

动态扩缩容：根据负载，线程数在 min 和 max 之间动态调整。

生命周期管理：线程空闲超时后会被回收，释放资源。

资源限制：通过 maxQueueSize 限制任务积压，防止内存无限增长。

优雅关闭：shutdown() 等待所有任务执行完成，shutdownNow() 强制中断。

解决 Promise 超高频创建

现代语言都有一个解决方案：对象池。

```
对象池
- 预创建对象
- 复用对象
- 重置状态
- 限制大小
```

```
对象池
- 预创建对象
- 复用对象
- 重置状态
- 限制大小
```

这两个方案的原理是，通过预创建对象，减少频繁的对象创建和销毁。

对象池的优点是，减少内存分配和垃圾回收的开销，提高性能。

对象池的缺点是，需要预分配内存，可能会浪费内存。

对象池的适用场景是，对象创建成本高，且对象可复用的情况